

Och

1 Introduction

After playing around with dependent types in an imperative programming setting during the masters thesis, I came to the conclusion that a fairly uncommon combination of language features would lead to an excellent combination of developer ergonomics and expressivity.

I want to study the most unusual + unweildly of these features in isolation with a core calculus before building full Ochre, specifically:

- The “terms are types” philosophy, as exonerated by PSS (Hutchins 2010). In Och this comes out roughly to “types are terms with holes”.
- Terms/types form a subtyping lattice, where supertypes can be freely substituted for any subtype without any coercions.

TODO: get rid of the “but why are these features important” feeling

1.1 Methodology

Och has been designed in an experimental fashion, where I come up with programs which should be rejected/accepted, then tweak the rules until the tests pass.

As a result of this

- Och is lacking a satisfying unifying theory and attempts of the soundness proof have been unsuccessful. This makes for a natural next step now that I have a set of rules which “mostly work”.
- There is quite an impressive set of test cases (see) which demonstrate that this idea has impressive expressivity both in terms of the complexity of the programs it as able to accept, and the subtlety of the bugs it is able to catch. **The most succinct answer to “why Och” is that I wanted these example programs to be handled correctly, and no system I could find does it quite to my liking.**

As a taster of what is to come: using Church/Scott encodings I have expressed booleans, natural numbers, finite sets (Fin n), pairs, lists, lists with compile-time-known-length, and safe list indexing.

These encodings use dependent elimination throughout, allowing the type system to catch bugs as subtle as adding the *wrong* two numbers, instead of just checking that the addition was well-shaped.

1.2 Goals

Soundness - Ochre is only valuable if programmers can trust the compiler to catch runtime errors. Proving the soundness of systems with the above features, let alone ownership + mutation is novel research as far as I can tell, Och is a place to do that research.

Expressivity - The type system must be able to articulate properties and catch bugs that people care about. E.g. it must be expressive enough to tell the difference between $x + y$ and $x + x$ if one is correct and the other incorrect.

Extensibility - Och is the first in a series of languages which will eventually culminate in full Ochre. As such, there is no point working with features which won’t be re-usable. This has already manifested in the decision to tackle arbitrary unbounded recursion via fix instead of having to add primitive inductive types and only well-founded induction.

2 Related Work

Pure Subtype Systems (Hutchins 2010) - Och was mostly developed before I found PSS, and to my delight PSS is an excellent execution on my core ideas, so I am in the process of re-framing Och as an *extension* of PSS instead of a new system. PSS explores the idea that “terms and types are the same thing” not just in terms of syntax but also semantics. The PSS paper is quite short, very easy to understand, and strongly recommended reading for anyone engaging with Och.

3 Language Semantics

This section describes “what does each piece of the language do” in natural language to give the reader an intuition for the objects involved before we get into the hard typing rules examples and metatheory.

Och syntax:

$e, \tau ::= x$	variable
$ \lambda(x \leq \tau).e$	lambda
$ e_1 e_2$	application
$ \top$	universe (top)
$ \perp$	primitive bottom
$ \iota(x \leq \tau).e$	self-type binder
$ \text{fix}(x \leq \tau).e$	recursive binder

There is no separate syntactic category for types: every τ above is itself a term.

Variables, lambdas, application as per the standard λ -calculus, except λ has a domain type annotation which is respected by the type checker.

$\top$ and $\perp$ are the top and bottom of the subtyping lattice. $\top$ represents the widest possible type, which tells you nothing about the term being typed, and $\perp$ is the empty type.

Fix $\text{fix}(x \leq \tau).e$ allows a term to be defined in terms of itself. x is the reference to self, and τ is the “upper bound” which the whole expression can be assumed to have while it’s being type checked (which prevents loops during type checking).

Iota $\iota(x \leq \tau).e$ allows a type to be defined in terms of **the term inhabiting it**. This means $e \sqsubseteq \iota(x : \tau_0).\tau_1$ is reduced to $e \sqsubseteq \tau_1[x \mapsto e]$ during checking (notice: the LHS has moved to the RHS). This is a Cedille-style self type (Fu and Stump 2014), and allows for church encodings with dependent elimination instead of having to add inductive datatypes to the language as a primitive.

3.1 Runtime semantics

You may think the runtime semantics of Och are unimportant since this is type systems research, but with how I’ve laid out the soundness proofs $\Downarrow$ plays a crucial role: soundness states “if a program type checks, it will succeed at runtime”, therefore $\Downarrow$ must *reject* ill-formed programs, otherwise our soundness becomes vacuously solvable (true on the RHS of an implication).

The concrete evaluator is a substitution-based call-by-value big-step interpreter on closed terms. $\{\lambda, \top, \perp\}$ are the only values. fix and ι eagerly unroll by substituting their self-reference into the body. Application is pure β -reduction — only lambdas can be applied.

Och has no concept of levels/universes/stages, so the type checker cannot enforce that $\{\top, \perp, \iota\}$ don’t appear at runtime. These are kept as values to avoid making soundness unprovable. Adding universes (see §3.6 of PSS (Hutchins 2010)) would let us erase type-level arguments during compilation, after which $\{\top, \perp, \iota\}$ would never appear at level 0.

We write the judgment $e \Downarrow v$ for “closed term e concretely evaluates to value v ”. Note: there is no context and no free/abstract variables: everything is eagerly substituted in.

$$\begin{array}{c}
\frac{}{v \Downarrow v} \text{E-VAL} \qquad \frac{f \Downarrow \lambda(x \leq \tau).b \quad a \Downarrow v_a \quad b[x \mapsto v_a] \Downarrow v}{f a \Downarrow v} \text{E-APP} \\
\\
\frac{b[x \mapsto \iota(x \leq \tau).b] \Downarrow v}{\iota(x \leq \tau).b \Downarrow v} \text{E-IOTA} \qquad \frac{b[x \mapsto \text{fix}(x \leq \tau).b] \Downarrow v}{\text{fix}(x \leq \tau).b \Downarrow v} \text{E-FIX}
\end{array}$$

where $v \in \text{Value} ::= \lambda(x \leq \tau).e$	lambda
$\perp$	primitive bottom
$\top$	universe (top)

The most important aspect of the runtime semantics are what is *missing* from the runtime semantics:

- Free variables are not values and have no rule, therefore they cannot occur at runtime, **which forces the type system to rule out ill-scoped variables.**
- Application can only apply to lambdas, **which forces the type system to verify applications are well typed w.r.t. the function domain.**

Fix and iota eagerly unroll — a program like $\text{fix}(x \leq \top).x$ loops forever (consuming fuel), which is the correct behaviour for non-terminating recursion. This lines up with what a programmer would expect to happen if they wrote `while True: pass` in Python or `let x = x in x` in Haskell.

4 Typing Rules

Och’s type system is structured around two relations:

Well-formedness $\Gamma \vdash e \text{ wf}$ states “ e is well-formed under Γ ” and is the “entry point” for the type system. Defined in Section 4.2. Delegates all type-comparison questions to subtyping.

Subtyping $S; \Gamma \vdash a \sqsubseteq b$ states “ a is a subtype of b under Γ , assuming coinductive hypotheses S ”. Subtyping in Och means set inclusion: $A \sqsubseteq B$ iff every value of A is also a value of B . Section 4.3 explains the S coinductive hypothesis system.

4.1 Context Γ

Γ is the typing context defined as $\Gamma ::= \emptyset \mid \Gamma, x \leq a$. Lookup is $\Gamma(x)$. Each entry describes what the type checker knows about a variable statically.

4.2 Well-formedness

The well-formedness judgment $\Gamma \vdash e \text{ wf}$ determines whether term e is well-formed under context Γ . In other systems this role would be served by a $\Gamma \vdash a : \tau$ rule which assigns a type to a term, but there is no need to assign a type to terms in Och because every term is already trivially its own type by [S-Refl].

Well-formedness validates purely structural conditions: annotations are types, binders are in scope, and applications have Π -typed functions with domain-inhabiting arguments.

$$\begin{array}{c}
\frac{x \in \text{dom}(\Gamma)}{\Gamma \vdash x \text{ wf}} \text{W-VAR} \qquad \frac{}{\Gamma \vdash \top \text{ wf}} \text{W-TYPE} \qquad \frac{}{\Gamma \vdash \perp \text{ wf}} \text{W-BOT} \\
\\
\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \lambda(x \leq A).b \text{ wf}} \text{W-LAM} \qquad \frac{\Gamma \vdash f \text{ wf} \quad \Gamma \vdash a \text{ wf} \quad \emptyset; \Gamma \vdash f \sqsubseteq \lambda(x \leq A).B \quad \emptyset; \Gamma \vdash a \sqsubseteq A}{\Gamma \vdash f a \text{ wf}} \text{W-APP} \\
\\
\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \iota(x \leq A).b \text{ wf}} \text{W-IOTA} \qquad \frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \leq A \vdash b \text{ wf}}{\Gamma \vdash \text{fix}(x \leq A).b \text{ wf}} \text{W-FIX}
\end{array}$$

[W-Var] requires variables be well-scoped.

[W-Lam], [W-Iota], [W-Fix] check that the annotation is well-formed and that the body is well-formed under the extended context.

[W-App] is the only rule with subtyping premises: the function must subtype some $\lambda(x \leq A).B$ (it has a Π -type), and the argument must subtype the domain A . No type is assigned to the result — the result’s type is the result itself (by [S-Refl]), and if the caller needs to know it subtypes something, that is a separate subtyping question.

4.3 Seen set S

Due to the prevalent usage of unbounded recursion in type definitions and encodings, many subtype checks involve infinite proof trees (e.g. $\emptyset; \emptyset \vdash \text{add one two} \sqsubseteq \text{Nat}$).

To derive these infinite trees in finite time Claude said I should use “the Brandt-Henglein device for equirecursive subtyping” as used in Brandt and Henglein (1998). Claude explains it as: “ S is a finite representation of a coinductive (potentially infinite) proof tree: when a goal recurs, the branch closes rather than recursing forever, encoding a regular infinite derivation as a finite one.”

As far as I can tell the reason it is remarkable enough to have a name is not that using it is particularly hard, but that convincing oneself it is a sound technique is hard, and Brandt/Henglein did just that. I am informed it only works when the cycles it closes contain at least one “productive” step, which are grouped together into Section 4.7.

The “real” subtyping judgment is $\emptyset; \Gamma \vdash a \sqsubseteq b$ (empty hypothesis set); non-empty S arises only inside a derivation.

4.4 Subtyping Rules

The subtyping rules fall into four categories. Each serves a distinct purpose; knowing which category a rule belongs to predicts its shape.

Category	Purpose	Rules	Extends S ?
Structural (Section 4.5)	Plumbing: recurse without inspecting constructors on either side	S-Refl, S-Top, S-BotL, S-Trans, S-Hyp, S-Var	no
Congruence (Section 4.6)	Match constructor on both sides; reduce to sub-obligations with variance	S-Lam, S-App-Cong, S-Iota-Cong, S-Fix-Cong	no
Productive unfolding (Section 4.7)	Unfold a recursive binder (i/fix); extend S ; enable coinductive closure	S-Iota-Intro, S-Unfold-Iota-L/R, S-Unfold-Fix-L/R	yes
Conversion (Section 4.8)	Close under head reduction so subtyping is closed under computation	S-Beta-L/R	no

Table 1: Rule taxonomy for declarative subtyping

The S -extension column matters: S-Hyp can only fire against entries that some ancestor productive rule installed, so any path to S-Hyp must cross an unfold — the productivity requirement made mechanical.

4.5 Structural rules

“Structural” rules talk about $\sqsubseteq$ as a relation on terms without inspecting either side’s head constructor: reflexivity, transitivity, the top element, hypothesis lookup, and variable lookup.

$$\begin{array}{lll}
\frac{}{S; \Gamma \vdash e \sqsubseteq e} \text{S-REFL} & \frac{S; \Gamma \vdash a \sqsubseteq b \quad S; \Gamma \vdash b \sqsubseteq c}{S; \Gamma \vdash a \sqsubseteq c} \text{S-TRANS} & \frac{(a, b) \in S}{S; \Gamma \vdash a \sqsubseteq b} \text{S-HYP} \\
\frac{}{S; \Gamma \vdash x \sqsubseteq \Gamma(x)} \text{S-VAR} & \frac{}{S; \Gamma \vdash e \sqsubseteq \top} \text{S-TOP} & \frac{}{S; \Gamma \vdash \perp \sqsubseteq e} \text{S-BOTL}
\end{array}$$

Ex falso via subsumption. There is no dedicated “absurd” eliminator. If $a \sqsubseteq \perp$ is derivable, then $a \sqsubseteq e$ for every e via [S-Trans] on [S-BotL]. The “contradiction” discharge is subsumption alone. This matches the DOT (Amin et al. 2012) tradition: $\perp$ inhabits every type trivially in subtyping, so any term whose type is already $\perp$ flows into any expected type without further ceremony.

Why [S-Trans] is a constructor, not a derived theorem. In a normal simply-typed subtyping relation, transitivity is admissible: a standard induction on the shape of the two derivations composes

them. That argument requires a decreasing syntactic measure — typically, both derivations get strictly smaller in each case of the composition. Och’s four unfold rules and iota-intro break this: unfolding $\text{fix}(x \leq A). \text{body}$ replaces it with $\text{body}[x \mapsto \text{fix}(x \leq A). \text{body}]$, which is *larger* than the original. There’s no obvious structural measure on derivations that decreases through an unfold, so transitivity is not derivable by induction on derivations. The seen-set discipline of Section 4.7 is the coinductive counterpart that keeps the relation consistent, but it doesn’t by itself give admissibility of transitivity.

Eliminating transitivity as a primitive rule (*transitivity elimination*) is highly desirable: it simplifies metatheory and makes the relation syntax-directed. PSS (Hutchins 2010) — a closely related system — did not manage to eliminate transitivity. Pasquale and García-Pérez (Pasquale and García-Pérez 2026) partially succeeded in a continuation of that work, but required switching the entire theory to a Krivine machine.

4.6 Congruence rules

Congruence rules do inspect the head constructor on both sides, *require it to match*, and reduce the goal to sub-obligations on the immediate sub-terms with appropriate variance.

$$\begin{array}{c}
\frac{S; \Gamma \vdash A_2 \sqsubseteq A_1 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \lambda(x \leq A_1).b_1 \sqsubseteq \lambda(x \leq A_2).b_2} \text{S-LAM-CONG} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash B_1 \sqsubseteq B_2}{S; \Gamma \vdash \iota(x \leq A_1).B_1 \sqsubseteq \iota(x \leq A_2).B_2} \text{S-IOTA-CONG} \\
\frac{S; \Gamma \vdash f_2 \sqsubseteq f_1 \quad S; \Gamma \vdash a_2 \sqsubseteq a_1}{S; \Gamma \vdash f_2 a_2 \sqsubseteq f_1 a_1} \text{S-APP-CONG} \\
\frac{S; \Gamma \vdash A_1 \sqsubseteq A_2 \quad S; \Gamma, x \leq A_2 \vdash b_1 \sqsubseteq b_2}{S; \Gamma \vdash \text{fix}(x \leq A_1).b_1 \sqsubseteq \text{fix}(x \leq A_2).b_2} \text{S-FIX-CONG}
\end{array}$$

The equivalence premise on [S-App-Cong] (both directions on the argument) is necessary because a neutral head can use its argument at any variance, so equivalence is the only sound congruence.

4.7 Productive unfolding (extend S)

A rule is *productive* when it replaces a goal whose head is a recursive binder (ι or fix) with a goal where that binder has been unfolded once. Unfolding makes the term *larger* in syntactic size, so no structural induction can traverse an arbitrary chain of unfolds. Productivity instead provides a *coinductive* handle: once at least one unfold has fired, the original goal is guaranteed to eventually re-appear as an ancestor, at which point [S-Hyp] can close the derivation.

The rules below are the only ones that extend S .

Let $S' = S, (a, b)$ where $S; \Gamma \vdash a \sqsubseteq b$ is the current goal.

$$\begin{array}{c}
\frac{S'; \Gamma \vdash a \sqsubseteq A \quad S'; \Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \leq A).B]}{S; \Gamma \vdash a \sqsubseteq \iota(x \leq A).B} \text{S-IOTA-INTRO} \\
\frac{S'; \Gamma \vdash B[x \mapsto \iota(x \leq A).B] \sqsubseteq c}{S; \Gamma \vdash \iota(x \leq A).B \sqsubseteq c} \text{S-UNFOLD-IOTA-L} \\
\frac{S'; \Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \leq A).B]}{S; \Gamma \vdash a \sqsubseteq \iota(x \leq A).B} \text{S-UNFOLD-IOTA-R} \\
\frac{S'; \Gamma \vdash b[x \mapsto \text{fix}(x \leq A).b] \sqsubseteq c}{S; \Gamma \vdash \text{fix}(x \leq A).b \sqsubseteq c} \text{S-UNFOLD-FIX-L} \\
\frac{S'; \Gamma \vdash a \sqsubseteq b[x \mapsto \text{fix}(x \leq A).b]}{S; \Gamma \vdash a \sqsubseteq \text{fix}(x \leq A).b} \text{S-UNFOLD-FIX-R}
\end{array}$$

S-Unfold-Iota-R is the weaker sibling of S-Iota-Intro (same conclusion, no annotation premise), needed to close the equivalence of ι -unfolding.

S-Iota-Intro is **the** rule which allows dependent elimination: it puts the thing being typed **into** the type, allowing the type to depend on the inhabitant.

Worked S-Iota-Intro example: $\text{true} \sqsubseteq \text{DBool}$. With

$$\begin{aligned}\text{true} &:= \lambda(P \leq \top).\lambda(t \leq \top).\lambda(f \leq \top).t \\ \text{false} &:= \lambda(P \leq \top).\lambda(t \leq \top).\lambda(f \leq \top).f \\ \text{DBool} &:= \text{fix}(B \leq \top).\iota(\text{self} \leq B).\lambda(P \leq B \rightarrow \top).\lambda(t \leq P \text{ true}).\lambda(f \leq P \text{ false}).P \text{ self}\end{aligned}$$

the constructors are plain lambdas with $\top$ domains — they carry no recursive reference to DBool . The subtyping check $\emptyset; \Gamma \vdash \text{true} \sqsubseteq \text{DBool}$ still requires the coinductive seen-set to close, because [S-Iota-Intro]’s annotation premise cycles back to the root goal:

$\begin{aligned}\emptyset; \Gamma \vdash \text{true} \sqsubseteq \text{DBool} \\ S_1; \Gamma \vdash \text{true} \sqsubseteq \iota(\text{self} \leq \text{DBool}).\lambda(P \leq \text{DBool} \rightarrow \top).\lambda(t \leq \\ P \text{ true}).\lambda(f \leq P \text{ false}).P \text{ self} \\ \text{where } S_1 = \{(\text{true}, \text{DBool})\} \\ \text{S-Iota-Intro} \text{ — annotation premise:} \\ S_1; \Gamma \vdash \text{true} \sqsubseteq \text{DBool} \\ \text{root goal reappears — closed by } (\text{true}, \text{DBool}) \in S_1 \\ \text{S-Iota-Intro} \text{ — body premise (after } [\text{self} \mapsto \text{true}]): \\ S_1; \Gamma \vdash \text{true} \sqsubseteq \lambda(P \leq \text{DBool} \rightarrow \top).\lambda(t \leq P \text{ true}).\lambda(f \leq P \text{ false}).P \text{ true} \\ \text{contravariant: } (\text{DBool} \rightarrow \top) \sqsubseteq \top \text{ by S-Top } \checkmark \\ \text{covariant body under } P \leq \text{DBool} \rightarrow \top: \\ S_1; \Gamma \vdash \lambda(t \leq \top).\lambda(f \leq \top).t \sqsubseteq \lambda(t \leq P \text{ true}).\lambda(f \leq P \text{ false}).P \text{ true} \\ \text{contravariant: } P \text{ true} \sqsubseteq \top \text{ by S-Top } \checkmark \\ \text{covariant body under } t \leq P \text{ true: S-Lam again} \\ \text{contravariant: } P \text{ false} \sqsubseteq \top \text{ by S-Top } \checkmark \\ \text{covariant body: } t \sqsubseteq P \text{ true by S-Var } (t \leq P \text{ true} \in \Gamma) \checkmark\end{aligned}$	root goal S-Unfold-Fix-R S-Hyp $\checkmark$ S-Lam S-Lam
---	--

Because the constructors have $\top$ domains, every contravariant [S-Lam] premise is discharged by [S-Top]. The only coinductive step is the [S-Iota-Intro] annotation premise, which closes via [S-Hyp] after one productive unfold. This is the Brandt–Henglein discipline in action: recursion in the subtyping judgment is legal *only across a productive unfold*, so non-productive loops (e.g. reflexivity-by-loop) cannot sneak through.

4.8 Conversion

The subtyping relation must be closed under head reduction: if a computes to a' , then $a \sqsubseteq b$ should hold iff $a' \sqsubseteq b$. These rules provide that closure.

$$\frac{S; \Gamma \vdash \text{body}[x \mapsto \arg] \sqsubseteq b}{S; \Gamma \vdash (\lambda(x \leq A). \text{body}) \arg \sqsubseteq b} \text{S-BETA-L} \qquad \frac{S; \Gamma \vdash a \sqsubseteq \text{body}[x \mapsto \arg]}{S; \Gamma \vdash a \sqsubseteq (\lambda(x \leq A). \text{body}) \arg} \text{S-BETA-R}$$

5 Example Programs

All encodings below are Church/Scott-style: data types are represented as their own eliminators, with no primitive inductive types in the language. Every definition is mechanised and tested in Lean 4 (see Section 7). This section builds from simple encodings to two programs that demonstrate Och’s ability to catch subtle bugs in dependently-typed code.

5.1 Booleans

$$\begin{aligned}\text{true} &:= \lambda(P \leq \top).\lambda(t \leq \top).\lambda(f \leq \top).t \\ \text{false} &:= \lambda(P \leq \top).\lambda(t \leq \top).\lambda(f \leq \top).f \\ \text{Bool} &:= \lambda(X \leq \top).\lambda(t \leq X).\lambda(f \leq X).X\end{aligned}$$

Bool is a standard Church-encoded boolean: the eliminator takes two branches and a return type X , and the type itself is X . Elimination is non-dependent — the return type X is the same regardless of whether the value is true or false.

$$\text{DBool} := \text{fix}(B \leq \top).\iota(\text{self} \leq B).\lambda(P \leq B \rightarrow \top).\lambda(t \leq P \text{ true}).\lambda(f \leq P \text{ false}).P \text{ self}$$

DBool is the *dependent* counterpart: its motive P is a function from values to types, so the return type $P \text{ self}$ varies with the value being eliminated. This is what ι enables — the self binder in $\iota(\text{self} \leq B)$

refers to the actual value inhabiting the type, and [S-Iota-Intro] substitutes it into P self during type checking.

Crucially, `Bool` and `DBool` *share the same constructors* $\{\text{true}, \text{false}\}$. The dependent elimination machinery lives entirely in the type (`DBool`'s ι and motive), not in the constructors. At runtime, a `DBool` value is identical to a `Bool` value — the difference is purely compile-time. This pattern recurs later with `Fin` reusing `Nat`'s constructors (Section 5.4).

5.2 Pairs

$$\begin{aligned}\text{Pair } A B &:= \lambda(X \leq \top). \lambda(k \leq A \rightarrow B \rightarrow X). X \\ \text{pair } A B a b &:= \lambda(X \leq \top). \lambda(k \leq A \rightarrow B \rightarrow X). k a b\end{aligned}$$

Church-encoded binary products. `Pair` projections `fst/snd` erase the unused component's type to $\top$, so call sites only supply the type of the component being projected.

5.3 Natural numbers

$$\begin{aligned}\text{zero} &:= \lambda(P \leq \top). \lambda(z \leq \top). \lambda(s \leq \top). z \\ \text{succ } n &:= \lambda(n \leq \top). \lambda(P \leq \top). \lambda(z \leq \top). \lambda(s \leq n \rightarrow \top). s n \\ \text{Nat} &:= \text{fix}(N \leq \top). \iota(\text{self} \leq N). \\ &\quad \lambda(P \leq N \rightarrow \top). \lambda(z \leq P \text{ zero}). \\ &\quad \lambda(s \leq \lambda(n \leq N). P (\text{succ } n)). P \text{ self}\end{aligned}$$

`Nat` uses both key features of `Och`. The outer `fix` ties the recursive knot (`Nat` refers to itself in the motive domain). The inner ι binds `self` to the value being typed, enabling *dependent* elimination: the return type $P \text{ self}$ varies with the value.

The constructors are plain lambdas with $\top$ domains — they carry no reference to `Nat`, so $\text{zero} \sqsubseteq \text{Nat}$ and $\text{succ } n \sqsubseteq \text{Nat}$ hold by the coinductive seen-set discipline (productive unfold of `fix`/ ι , then all contravariant domain checks discharge via [S-Top]).

Addition is defined with an explicit `fix` since the Scott-style eliminator provides no induction hypothesis:

$$\begin{aligned}\text{add} &:= \text{fix } \text{add} \leq \text{Nat} \rightarrow \text{Nat} \rightarrow \text{Nat}. \\ &\quad \lambda(n \leq \text{Nat}). \lambda(m \leq \text{Nat}). \\ &\quad n (\lambda(_ \leq \text{Nat}). \text{Nat}) m (\lambda(\text{pred} \leq \text{Nat}). \text{succ } (\text{add } \text{pred } m))\end{aligned}$$

5.4 Finite sets

$$\begin{aligned}\text{Fin} &:= \text{fix}(F \leq \text{Nat} \rightarrow \top). \lambda(n \leq \text{Nat}). \\ &\quad n (\lambda(_ \leq \text{Nat}). \top) \perp \\ &\quad (\lambda(\text{pred} \leq \text{Nat}). \iota(\text{self} \leq \text{Nat}). \\ &\quad \quad \lambda(P \leq \text{Nat} \rightarrow \top). \lambda(\text{fz} \leq P \text{ zero}). \\ &\quad \quad \lambda(\text{fs} \leq \lambda(q \leq F \text{ pred}). P (\text{succ } q)). P \text{ self})\end{aligned}$$

`Fin` n is the type of naturals strictly less than n . The zero-length case is $\perp$ (primitive bottom), so `Fin` `zero` is uninhabited. The successor case is an ι -type whose `self` is a `Nat` — this means every `Fin` value is automatically a `Nat` value ($\text{Fin } n \sqsubseteq \text{Nat}$). **(I consider this a majorly cool result)**

Natural number literals inhabit `Fin` by subsumption: $\text{zero} \sqsubseteq \text{Fin } (\text{succ } n)$ holds for any n , and $n \sqsubseteq \text{Fin } n$ is correctly rejected (the diagonal). No separate `FZ`/`FS` constructors are needed.

5.5 Length-indexed arrays

$$\begin{aligned}\text{Array} &:= \text{fix}(\text{Arr} \leq \text{Nat} \rightarrow \top \rightarrow \top). \lambda(n \leq \text{Nat}). \lambda(T \leq \top). \\ &\quad n (\lambda(_ \leq \text{Nat}). \top) \text{Unit } (\lambda(\text{pred} \leq \text{Nat}). \text{Pair } T (\text{Arr } \text{pred } T))\end{aligned}$$

Array $n\ T$ computes by eliminating the length index: $\text{Array zero } T \equiv \text{Unit}$ and $\text{Array (succ } k) T \equiv \text{Pair } T (\text{Array } k T)$. Arrays are built with `unit/pair` directly.

Vectors package a length with an array of that length, using a dependent pair (`Sigma`):

$$\text{Vec } T := \text{Sigma Nat } (\lambda(n \leq \text{Nat}). \text{Array } n\ T)$$

5.6 appendVec: catching the wrong addition

The north-star example. `appendVec` unpacks two vectors, concatenates their arrays with `appendArrays`, and repacks the result with the summed length:

$$\begin{aligned} \text{appendVec} &:= \lambda(T \leq \top). \lambda(v_1 \leq \text{Vec } T). \lambda(v_2 \leq \text{Vec } T). \\ &\quad v_1 (\text{Vec } T) (\lambda(n_1 \leq \text{Nat}). \lambda(\text{arr}_1 \leq \text{Array } n_1\ T). \\ &\quad \quad v_2 (\text{Vec } T) (\lambda(n_2 \leq \text{Nat}). \lambda(\text{arr}_2 \leq \text{Array } n_2\ T). \\ &\quad \quad \text{mkVec } T (\text{add } n_1\ n_2) (\text{appendArrays } T\ n_1\ n_2\ \text{arr}_1\ \text{arr}_2))) \end{aligned}$$

This type-checks: the length `add  $n_1\ n_2$` matches the length of the concatenated array.

Now consider a version with a deliberate bug — `add  $n_1\ n_1$` instead of `add  $n_1\ n_2$` :

$$\text{appendVec}_{\text{wrong}} := \dots \text{mkVec } T (\text{add } n_1\ n_1) (\text{appendArrays } T\ n_1\ n_2\ \text{arr}_1\ \text{arr}_2) \dots$$

The type checker *rejects* this: $\text{arr}_2 \sqsubseteq \text{Array } n_1\ T$ fails because arr_2 has type $\text{Array } n_2\ T$ and $n_2 \neq n_1$. The system catches a bug as subtle as adding the wrong two numbers.

5.7 indexArr: safe array indexing

`indexArr` looks up the i -th element of an $\text{Array } n\ T$, where i has type $\text{Fin } n$:

$$\begin{aligned} \text{indexArr} &:= \text{fix } (\text{self} \leq (\lambda(T \leq \top). \lambda(n \leq \text{Nat}). \text{Array } n\ T \rightarrow \text{Fin } n \rightarrow T)). \\ &\quad \lambda(T \leq \top). \lambda(n \leq \text{Nat}). n (\lambda(m \leq \text{Nat}). \text{Array } m\ T \rightarrow \text{Fin } m \rightarrow T) \\ &\quad \quad (\lambda(\text{arr} \leq \text{Array zero } T). \lambda(i \leq \text{Fin zero}). i) \\ &\quad \quad (\lambda(p \leq \text{Nat}). \lambda(\text{arr} \leq \text{Array (succ } p) T). \lambda(i \leq \text{Fin (succ } p)). \dots) \end{aligned}$$

In the zero-length branch, i has type $\text{Fin zero} = \perp$, so the branch body is i itself — `ex falso` via `[S-BotL]`. In the successor branch, the array is destructured as a pair and i eliminates to either the head or a recursive call on the tail.

The payoff is at call sites:

$$\begin{aligned} \text{indexArr Nat three arr zero} &\quad \checkmark \quad (\text{zero} \sqsubseteq \text{Fin three}) \\ \text{indexArr Nat three arr two} &\quad \checkmark \quad (\text{two} \sqsubseteq \text{Fin three}) \\ \text{indexArr Nat three arr three} &\quad \times \quad (\text{three} \not\sqsubseteq \text{Fin three}) \end{aligned}$$

Out-of-bounds access is a *compile-time* error: the index literal fails to subtype $\text{Fin } n$, and no runtime check is needed.

6 Metatheory

Soundness connects the concrete semantics (Section 3.1), subtyping (Section 4.4), and well-formedness (Section 4.2).

Evaluation equivalence. If e evaluates to e' , both directions of subtyping hold.

$$e \Downarrow e' \text{ implies } \emptyset; \emptyset \vdash e' \sqsubseteq e \text{ and } \emptyset; \emptyset \vdash e \sqsubseteq e'$$

Progress. A well-formed closed term doesn't get stuck during evaluation. The evaluator may return a value or run out of fuel but never reaches an error state.

End-to-end. Composing the above: if e is well-formed and subtypes τ , and e evaluates to e' , the result subtypes τ .

$$\emptyset \vdash e \text{ wf and } \emptyset; \emptyset \vdash e \sqsubseteq \tau \text{ and } e \Downarrow e' \text{ implies } \emptyset; \emptyset \vdash e' \sqsubseteq \tau$$

6.1 What soundness promises to the programmer

Two separable runtime properties a type system can promise:

1. **Termination.** Running a well-typed program eventually produces a value. A totality / normalization claim.
2. **No runtime type errors.** Conditional on the program *not* diverging, the value it produces is well-formed and the program never reaches a stuck state.

Och **does not aim to prove (1)**. Consistency and normalization are explicitly deferred; the calculus admits non-terminating terms by design (type-in-type; general recursion via fix).

Och **does aim to prove (2)**, and this is the real runtime guarantee of the type system. The language expects the following discipline: **type-check first; only evaluate if it succeeded**.

7 Lean Formalisation

The calculus described above has been formalised in Lean 4, which is used to maintain a test suite of example programs and allow AI agents to iterate on the metatheory. The source is available at github.com/charlielidbury/ochre.

Bibliography

- Amin, Nada, Adriaan Moors, and Martin Odersky. 2012. “Dependent Object Types.” In “Workshop on Foundations of Object-Oriented Languages (Fool).” Special issue, *Workshop on Foundations of Object-Oriented Languages (FOOL)* (Tucson, AZ, USA),. <https://infoscience.epfl.ch/server/api/core/bitstreams/678b1818-06aa-47f1-b7e0-783413d76044/content>.
- Brandt, Michael, and Fritz Henglein. 1998. “Coinductive Axiomatization of Recursive Type Equality and Subtyping.” *Fundamenta Informaticae* 33 (4): 309–38. https://doi.org/10.1007/3-540-62688-3_29.
- Fu, Peng, and Aaron Stump. 2014. “Self Types for Dependently Typed Lambda Encodings.” In “Rewriting and Typed Lambda Calculi (RTA-Tlca).” Special issue, *Rewriting and Typed Lambda Calculi (RTA-TLCA)*,. <https://homepage.divms.uiowa.edu/~astump/papers/fu-stump-rta-tlca-14.pdf>.
- Hutchins, DeLesley S. 2010. “Pure Subtype Systems.” In “Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (Popl).” Special issue, *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 287–98. <https://doi.org/10.1145/1706299.1706334>.
- Pasquale, Valentin, and Álvaro García-Pérez. 2026. “Towards the Type Safety of Pure Subtype Systems.” In “34th EACSL Annual Conference on Computer Science Logic (Csl).” Special issue, *34th EACSL Annual Conference on Computer Science Logic (CSL)*, Leibniz International Proceedings in Informatics (LIPIcs), vol. 363 : 37:1–16. <https://doi.org/10.4230/LIPIcs.CSL.2026.37>.